University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

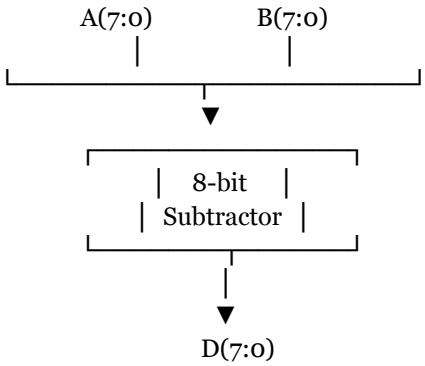
Mid-Term Lab Examination | Group C2 | Spring 2026

Duration	30 minutes	**Total Marks**	20
Student Name	_____	**ID**	_____

Mid-Term Task: 8-bit Subtractor

Objective. Design and implement an 8-bit subtractor in VHDL using the subtractor concepts developed in the laboratory. The circuit must perform the operation D = A − B, where A and B are 8-bit binary inputs and D is the 8-bit difference output.

System Architecture



Functional Requirement. For every applied pair of inputs, the circuit must produce the 8-bit difference D = A − B.

Required Test Cases

Use the following cases in your testbench and verify the expected 8-bit output.

Test	**A**	**B**	**Operation**	**Expected D**
1	00001111	00000011	15 − 3	00001100
2	00101010	00001010	42 − 10	00100000
3	01010101	00110011	85 − 51	00100010
4	11111111	00000001	255 − 1	11111110
5	10000000	00000001	128 − 1	01111111

Implementation Requirements

- Use VHDL and IEEE.STD_LOGIC_1164.ALL.
- The design should be structural/modular and should demonstrate the subtractor components covered in the laboratory.
- Do not use a single behavioral arithmetic statement such as D <= A - B; as the complete implementation.
- Create a separate testbench and generate appropriate input stimulus.
- Display A, B, and D in the simulation waveform.
- No clock or register is required for this task; the required circuit is combinational.

Submission

While submitting to the classroom prepare the report from the third page of this template including the current template in the first and second page. From the third page write your report including but not limited to the following points.

- Attach your entity code and testbench code
- Attach your output visuals
- Challenges faced during the experiment
- Conclusion

• Attach your entity code and testbench code

This is entity code:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity subtractor_8bit is
   Port ( A         : in  STD_LOGIC_VECTOR (7 downto 0); -- 8-bit Minuend
          B         : in  STD_LOGIC_VECTOR (7 downto 0); -- 8-bit Subtrahend
          Diff      : out STD_LOGIC_VECTOR (7 downto 0); -- 8-bit Difference
          Borrow_out : out STD_LOGIC                     -- Final active-low borrow flag
        );
end subtractor_8bit;

architecture Structural of subtractor_8bit is

   -- Declare your 4-bit subtractor module as a component
   component subtractor_4bit is
      Port ( A    : in  STD_LOGIC_VECTOR (3 downto 0);
             B    : in  STD_LOGIC_VECTOR (3 downto 0);
             Cin  : in  STD_LOGIC;
             Diff : out STD_LOGIC_VECTOR (3 downto 0);
             Cout : out STD_LOGIC
           );
   end component;

   -- Internal signal to link the lower 4-bit block to the upper 4-bit block
   signal middle_carry : STD_LOGIC;

begin

   -- 1. Lower 4-bit Subtractor (Bits 0 to 3)
   -- We feed '1' to its Cin to kickstart the two's complement arithmetic (+1)
   SUB_LOW: subtractor_4bit
      port map (
         A    => A(3 downto 0),
         B    => B(3 downto 0),
         Cin  => '1',
         Diff => Diff(3 downto 0),
         Cout => middle_carry
      );

   -- 2. Upper 4-bit Subtractor (Bits 4 to 7)
   -- It takes the output carry from the lower block and forwards the final stage out
   SUB_HIGH: subtractor_4bit
      port map (
```

```
        A    => A(7 downto 4),
        B    => B(7 downto 4),
        Cin  => middle_carry,
        Diff => Diff(7 downto 4),
        Cout => Borrow_out
    );

end Structural;
```

This is testbench code:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_subtractor_8bit is
end tb_subtractor_8bit;

architecture Behavioral of tb_subtractor_8bit is

  -- Component Declaration for the Unit Under Test (UUT)
  component subtractor_8bit is
    Port ( A        : in  STD_LOGIC_VECTOR (7 downto 0);
           B        : in  STD_LOGIC_VECTOR (7 downto 0);
           Diff     : out STD_LOGIC_VECTOR (7 downto 0);
           Borrow_out : out STD_LOGIC
         );
  end component;

  -- Inputs
  signal A : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
  signal B : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');

  -- Outputs
  signal Diff     : STD_LOGIC_VECTOR(7 downto 0);
  signal Borrow_out : STD_LOGIC;

begin

  -- Instantiate the Unit Under Test (UUT)
  uut: subtractor_8bit port map (
      A => A,
      B => B,
      Diff => Diff,
      Borrow_out => Borrow_out
    );

  -- Stimulus process
```

```
    stim_proc: process
    begin
        -- Wait 100 ns for global reset
        wait for 100 ns;

        -- Test Case 1: 100 - 30 = 70
        A <= std_logic_vector(to_unsigned(100, 8)); -- "01100100"
        B <= std_logic_vector(to_unsigned(30, 8));  -- "00011110"
        wait for 50 ns;

        -- Test Case 2: 255 - 1 = 254
        A <= "11111111";
        B <= "00000001";
        wait for 50 ns;

        -- Test Case 3: 10 - 20 = -10 (Underflow case)
        -- Expected Borrow_out will be '0' indicating B > A
        A <= std_logic_vector(to_unsigned(10, 8));
        B <= std_logic_vector(to_unsigned(20, 8));
        wait for 50 ns;

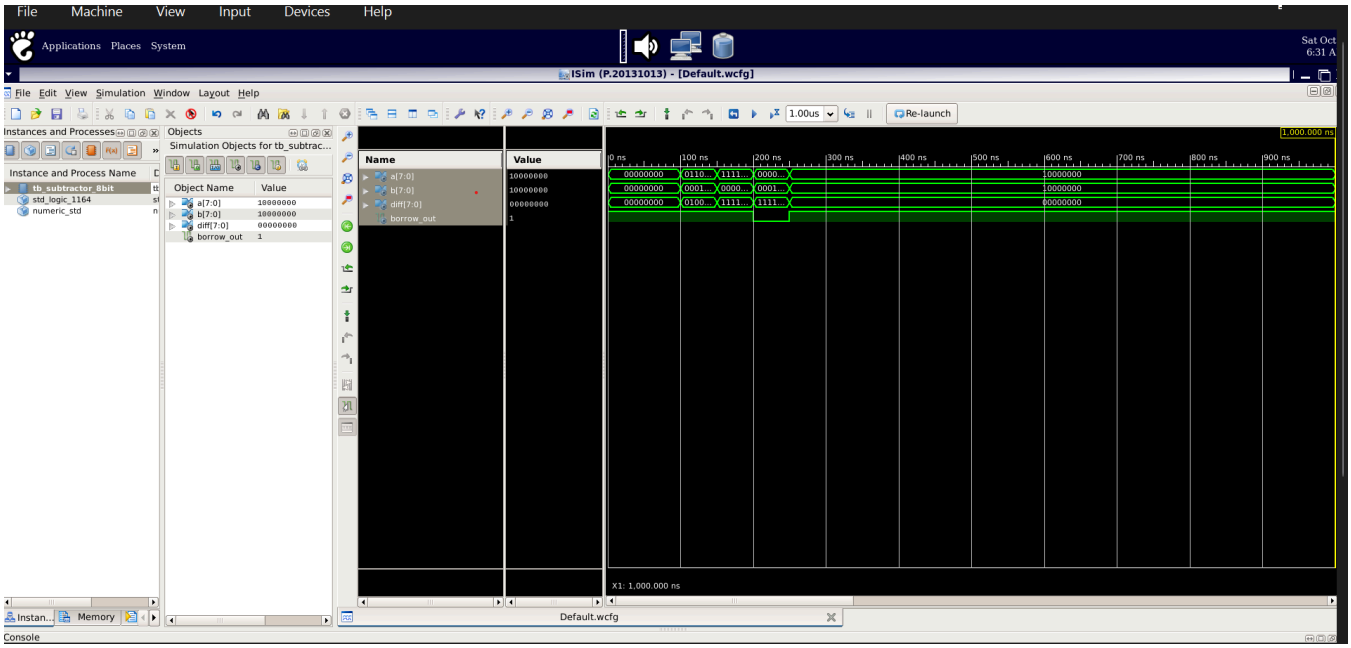
        -- Test Case 4: 128 - 128 = 0
        A <= "10000000";
        B <= "10000000";
        wait for 50 ns;

        -- End simulation safely
        wait;
    end process;

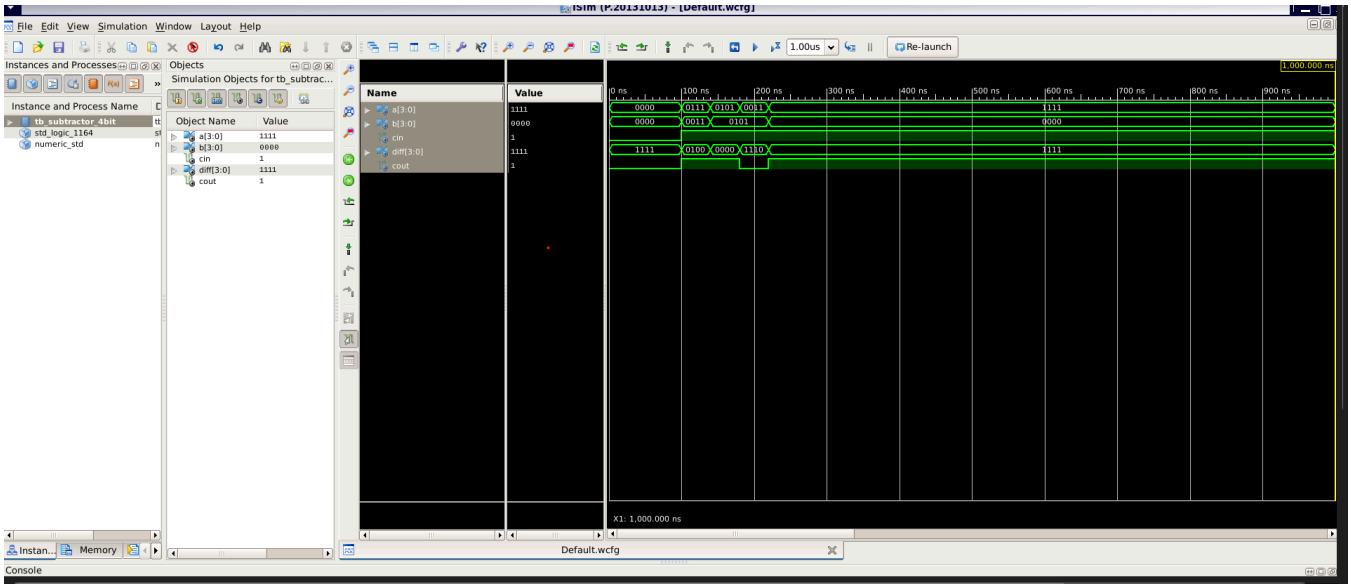
end Behavioral;
```

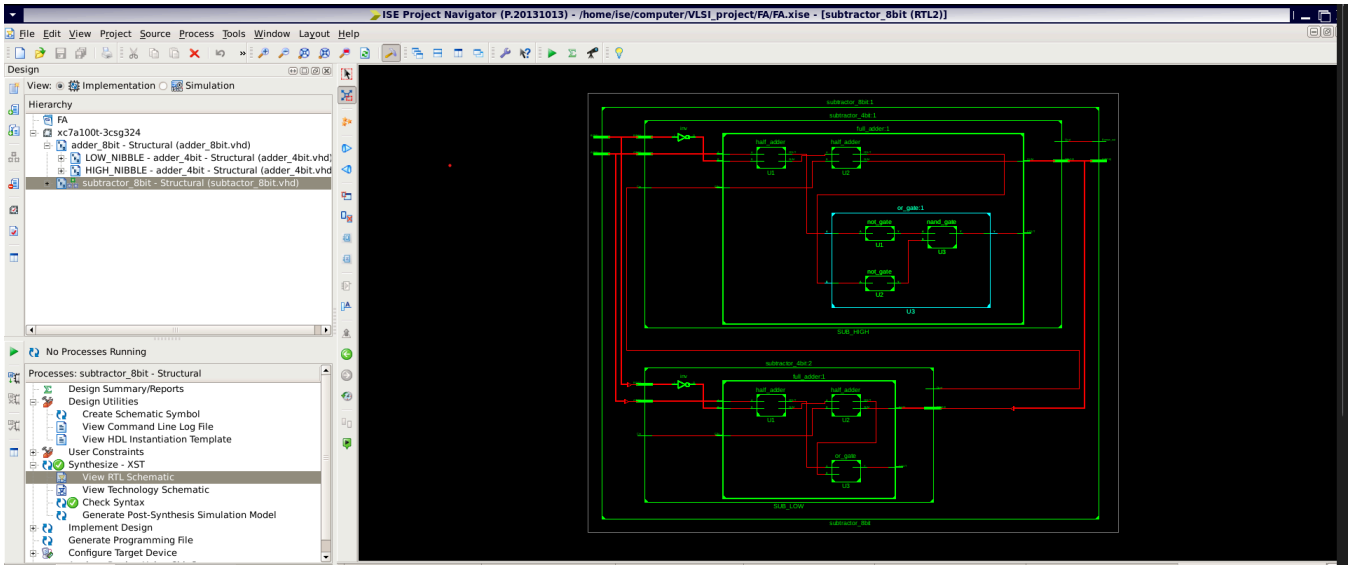
• Attach your output visuals

Tb 8 sub behaviour



Tb 4 sub behaviour





Challenges Faced During the Experiment

The main challenge was choosing the subtraction method. I used two's complement subtraction, A − B = A + NOT(B) + 1, because it allowed me to reuse the full-adder modules from the lab. The initial carry-in was set to '1' for the +1.

Another challenge was understanding the borrow output. In two's complement subtraction, the final carry-out is '1' when A ≥ B and '0' when A < B, so it works opposite to a normal borrow signal. I also had to carefully connect the two 4-bit blocks and match their port widths and bit ranges.

Since behavioral subtraction (A - B) was not allowed, all operations were implemented using full-adder components. Finally, checking the waveform in ISim and managing the experiment within the time limit were also challenging.

Conclusion

An 8-bit subtractor was successfully designed in VHDL using two 4-bit subtractor blocks and full-adder components. Two's complement was used for subtraction, with an initial carry-in of '1'. The RTL schematic and ISim waveform confirmed that the design worked correctly, including the borrow indication. This experiment helped me understand how smaller modules can be combined to build a larger combinational circuit.